A Multi-Dimensional, Per-Pass Empirical Study of the LLVM Optimization Pipeline

Ph.D. Student's Activity Report

Federico Bruzzone

Computer Science Department, Università degli Studi di Milano
Supervisor: Prof. **Walter Cazzola**
Email: federico.bruzzone@unimi.it
Website: federicobruzzone.github.io
Slides: federicobruzzone.github.io/presentations/llvm-optimization-pipeline.pdf

22 June 2026

Optimizing compilers

Since the dawn of computing, compiler optimizations have long bridged high-level abstractions and efficient machine code [**? ? ? ? ? ?**].

Optimizing compilers [**? ?**] use analysis passes to guide transformations [**? ? ?**] that enhance performance without altering observable behavior [**? ? ?**]—modern architectures (e.g., x86-64, AArch64) make this an interplay of interdependent passes [**?**].

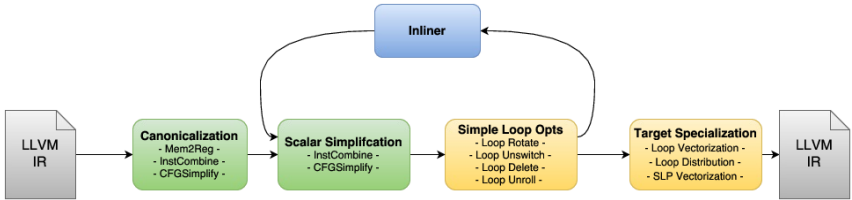
LLVM [**?**] exemplifies this complexity, serving as the standard backend for C/C++, Rust [**?**], Julia [**?**], and via **MLIR** [**?**]—for *machine learning* (ML) and *deep learning* (DL) stacks [**? ?**] that rely on ML/DL optimizing compilers [**? ? ? ?**] (e.g., NVCC[1], OpenXLA [**?**], TVM [**?**]) and automotive pipelines.[2]

[1] developer.nvidia.com/cuda-llvm-compiler

[2] Tesla Full Self-Driving v14.3: stats.tessie.com/versions/2026.2.9.6

LLVM Optimization Pipeline

LLVM is a modular SSA-based compilation framework [**? ? ?**]. Its `opt` tool applies sequences of analysis and transformation passes, forming the core optimization pipeline.



It comprises parameterized nested managers (`module`, `function`, `cgscc`, `loop`, `loop-mssa`), each scoped to a specific IR granularity.

We need better optimization evaluation!

Per-pass impact analysis is critical for compiler engineers and end-users [**? ? ?**] to:

- ☞ address *undecidable* [**? ? ?**] phase ordering challenges [**? ? ?**],
- ☞ improve selection heuristics and ML-based auto-tuning [**? ? ? ? ? ? ?**],
- ☞ guide new optimization design, and
- ☞ understand pass–hardware interactions [**?**].

However, existing work evaluates LLVM optimizations in isolation or focuses mainly on execution time [**?**], overlooking multi-dimensional trade-offs—a pass improving runtime may increase binary size or hurt cache locality.

Moreover, the debate over hand-written GPU kernels vs. ML-compiler-optimized kernels remains open [**?**].[3]

[3] octoml-optimizes-apache-tvm-for-apples-m1-beats-core-ml-4-by-29

A Study to Rule Them All (150+ studies at ICSE'26 A*)

We address this gap with a multi-dimensional, systematic empirical study of the LLVM `-O3` pipeline, measuring not only **runtime** but also **compile time**, **binary size**, and a broad set of **HW performance counters**[4] similar to the methodology of Chen *et al.* [**?**].

> **RQ$_1$.** *What is the marginal performance impact of individual passes within the LLVM* `-O3` *optimization pipeline?*

> **RQ$_2$.** *To what extent do optimization passes trade off execution speed, compilation overhead, and binary footprint?*

> **RQ$_3$.** *How do IR-level transformations affect the evolution of HW performance counters across optimization pipelines?*

> **RQ$_4$.** *What fraction of achievable speedup is lost to intra-pipeline interference, and which pass drives the loss?*

[4] E.g., those exposed by Model-Specific Registers (MSRs) on x86 CPUs: cache misses, `cycles/instructions` for ILP, and energy consumption [**? ? ?**] via RAPL.

A Study to Rule Them All (150+ studies at ICSE'26 A*)

We address this gap with a multi-dimensional, systematic empirical study of the LLVM `-O3` pipeline, measuring not only **runtime** but also **compile time**, **binary size**, and a broad set of **HW performance counters**[4] similar to the methodology of Chen *et al.* [**?**].

RQ$_1$. *What is the marginal performance impact of individual passes within the LLVM* `-O3` *optimization pipeline?*

RQ$_2$. *To what extent do optimization passes trade off execution speed, compilation overhead, and binary footprint?*

RQ$_3$. *How do IR-level transformations affect the evolution of HW performance counters across optimization pipelines?*

RQ$_4$. *What fraction of achievable speedup is lost to intra-pipeline interference, and which pass drives the loss?*

[4] E.g., those exposed by Model-Specific Registers (MSRs) on x86 CPUs: cache misses, `cycles/instructions` for ILP, and energy consumption [**? ? ?**] via RAPL.

A Study to Rule Them All (150+ studies at ICSE'26 A*)

We address this gap with a multi-dimensional, systematic empirical study of the LLVM `-O3` pipeline, measuring not only **runtime** but also **compile time**, **binary size**, and a broad set of **HW performance counters**[4] similar to the methodology of Chen *et al.* [**?**].

RQ$_1$. *What is the marginal performance impact of individual passes within the LLVM* `-O3` *optimization pipeline?*

RQ$_2$. *To what extent do optimization passes trade off execution speed, compilation overhead, and binary footprint?*

RQ$_3$. *How do IR-level transformations affect the evolution of HW performance counters across optimization pipelines?*

RQ$_4$. *What fraction of achievable speedup is lost to intra-pipeline interference, and which pass drives the loss?*

[4] E.g., those exposed by Model-Specific Registers (MSRs) on x86 CPUs: cache misses, `cycles/instructions` for ILP, and energy consumption [**? ? ?**] via RAPL.

A Study to Rule Them All (150+ studies at ICSE'26 A*)

We address this gap with a multi-dimensional, systematic empirical study of the LLVM `-O3` pipeline, measuring not only **runtime** but also **compile time**, **binary size**, and a broad set of **HW performance counters**[4] similar to the methodology of Chen *et al.* [**?**].

RQ$_1$. *What is the marginal performance impact of individual passes within the LLVM* `-O3` *optimization pipeline?*

RQ$_2$. *To what extent do optimization passes trade off execution speed, compilation overhead, and binary footprint?*

RQ$_3$. *How do IR-level transformations affect the evolution of HW performance counters across optimization pipelines?*

RQ$_4$. *What fraction of achievable speedup is lost to intra-pipeline interference, and which pass drives the loss?*

[4] E.g., those exposed by Model-Specific Registers (MSRs) on x86 CPUs: cache misses, `cycles/instructions` for ILP, and energy consumption [**? ? ?**] via RAPL.

A Study to Rule Them All (150+ studies at ICSE'26 A*)

We address this gap with a multi-dimensional, systematic empirical study of the LLVM `-O3` pipeline, measuring not only **runtime** but also **compile time**, **binary size**, and a broad set of **HW performance counters**[4] similar to the methodology of Chen *et al.* [**?**].

RQ$_1$. *What is the marginal performance impact of individual passes within the LLVM* `-O3` *optimization pipeline?*

RQ$_2$. *To what extent do optimization passes trade off execution speed, compilation overhead, and binary footprint?*

RQ$_3$. *How do IR-level transformations affect the evolution of HW performance counters across optimization pipelines?*

RQ$_4$. *What fraction of achievable speedup is lost to intra-pipeline interference, and which pass drives the loss?*

[4] E.g., those exposed by Model-Specific Registers (MSRs) on x86 CPUs: cache misses, `cycles/instructions` for ILP, and energy consumption [**? ? ?**] via RAPL.

Experimental setup

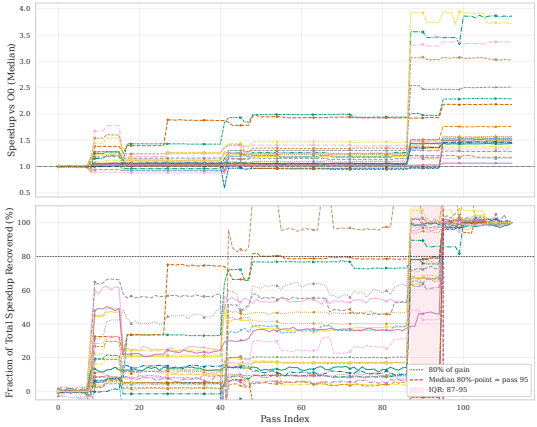
All experiments ran on Intel Core i9-12900KF:

- ☞ Alder Lake (8P+8E cores, 5.2 GHz max);
- ☞ 128 GB RAM;
- ☞ Fedora Linux 43 (kernel 6.19);
- ☞ Cache hierarchy: 640 KiB **L1-D**, 768 KiB **L1-I**, 14 MiB **L2**, 30 MiB **L3**.

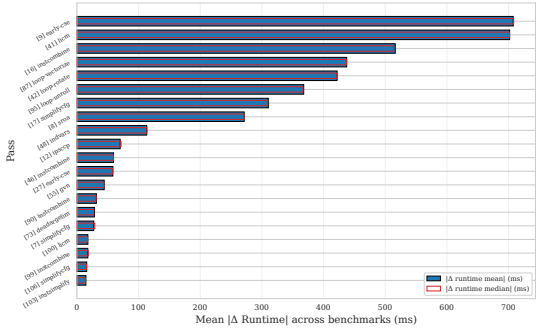
We used LLVM 21.1.8 compiled in release, with no assertions, *profile-guided optimizations* (PGO) disabled, studying the `-O3` pipeline.

Runtime uses `hyperfine` (3 warmup, 15 timed runs), counters use `perf stat` (2 warmup, 10 timed) in a separate batch—10 runs suffice for CI95 below 1% on all counter metrics.

Cumulative speedup, normalised gain, and top-20 passes



$$G_b^i = (S_b^i - 1)/(S_b^* - 1) \in [0, 1]$$



- Pareto-skewed distribution: the top decile accounts for the majority of impact.
- The ranking is dominated by `early-cse`, `licm`, `instcombine`, `loop-vectorize`, `loop-rotate`, `loop-unroll`, `simplifycfg`, `sroa`, `indvars`.

`2mm` Linear-Algebra Kernel from PolyBench [?][5]

A double matrix-matrix multiplication (GEMM) in $O(N^3)$ time.

```
/* D = a*A*B*C + b*D */
for (i = 0; i < NI; i++)
 for (j = 0; j < NJ; j++)
  for (k = 0; k < NK; k++)
   t[i][j] += a*A[i][k]*B[k][j];
for (i = 0; i < NI; i++)
 for (j = 0; j < NL; j++) {
  D[i][j] *= b;
  for (k = 0; k < NJ; k++)
   D[i][j] += t[i][k]*C[k][j];
 }
```

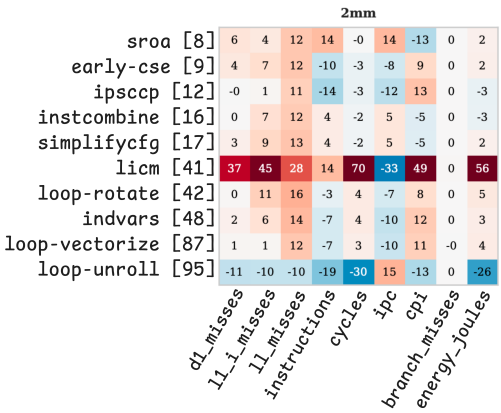
Pratical utility of the kernel:

- **Data Reuse**: CPU/GPU spatial and temporal cache locality.
- **Pipeline Parallelism**: CPU/GPU SIMD, *instruction level* parallelism (ILP), and superscalar pipelined execution.
- **Deep Learning**: the basic block during inference (and training) in neural networks is `2mm`.
- **Features**: *affine* perfect (first) and imperfect (second) loop nests suitable for polyhedral optimizations.

[5] github.com/MatthiasJReisinger/PolyBenchC-4.2.1/polybench.pdf

HW-counter signatures at the top-10 passes (2mm)

The blue-colored boxes are **improvements** over the baseline (`-O0`), while red-colored are **regressions**.



- ☞ The branch predictor is not waiting: `branch_misses` are 0 everywhere! Thanks to prefetching due to constant stride.
- ☞ `licm` is introducing lots of regressions, but recovered later.
- ☞ `loop-unroll` decreased `d1`, `l1_i`, and `l1_d` counters. The energy consumption is also decreasing.
- ☞ `loop-vectorize` is not soo significant, but it is still worth mentioning.

Takeaways

- ☞ The top-10 passes are responsible for the majority of the performance impact.
- ☞ Measuring only execution time is insufficient to understand the trade-offs between passes.
- ☞ HW counters provide a more detailed view of the impact of passes on the underlying architecture.
- ☞ Studying LLVM optimizations is important for both general-purpose and domain-specific programming languages, including AI/ML/DL workloads.

Thank you!

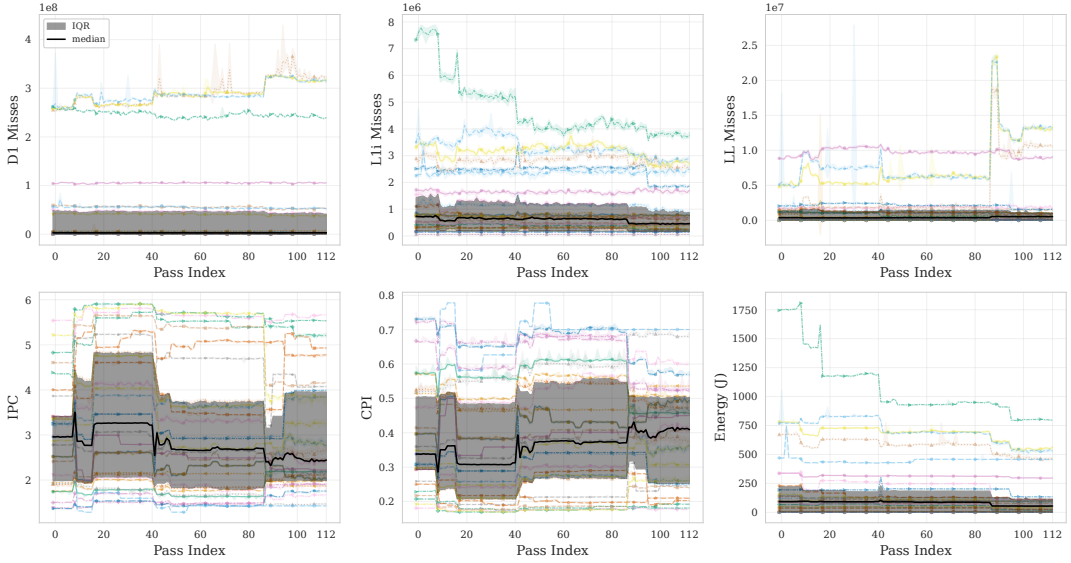
Questions?

federicobruzzone.github.io

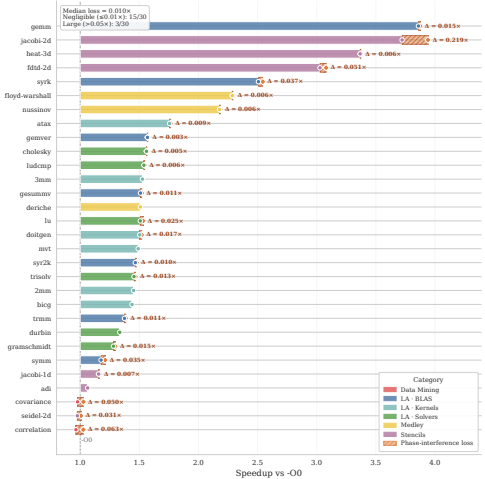
Slides: federicobruzzone.github.io/presentations/llvm-optimization-pipeline.pdf

References I

HW counters aggregated across the PolyBench suite



Bounded phase-interference loss



We **define** a novel optimistic additive bound:

$$S_{\text{opt}} = \sum_i \max(\Delta S_i, 0),$$

$$L = \frac{S_{\text{opt}} - [S_{\text{actual}}]_0^{S_{\text{opt}}}}{S_{\text{opt}}},$$

where $[x]_0^{S_{\text{opt}}} = \min(\max(x, 0), S_{\text{opt}})$ and $S_{\text{actual}} = S_{\text{final}} - 1$.

L: mean 46.35%!

Why ∼46%, despite tiny ΔS_i?

Marginals telescope: with $\Delta S_i = S_i - S_{i-1}$ and $S_0 = 1$, $\sum_i \Delta S_i = S_{\text{final}} - 1 = S_{\text{actual}}$.

So, splitting into constructive (P) and destructive (N) movement,

$$P = \sum_i \max(\Delta S_i, 0) = S_{\text{opt}}, \qquad N = \sum_i |\min(\Delta S_i, 0)|,$$

the bound collapses (away from the clamp) to a **pure ratio**:

$$L = \frac{S_{\text{opt}} - S_{\text{actual}}}{S_{\text{opt}}} = \frac{P - (P - N)}{P} = \boxed{\frac{N}{P}}.$$

Why ∼46%, despite tiny ΔS_i? (Cont.)

L is scale-invariant

L is a *ratio*, not an amplitude: scale every ΔS_i by 1000 or by 0.001 and L does **not** change. Only the *balance* destructive/constructive matters — never the magnitude.

No catastrophic pass needed

$|\Delta S_i| \approx 0.2\text{–}0.3\%$ over **113 passes**:

$$\underbrace{S_{\text{opt}} \approx 9\%}_{P} \quad \text{erasing} \quad \underbrace{6\%}_{N} \quad \Rightarrow \quad L \approx 67\%.$$

Micro-deltas *accumulate*.

$\Rightarrow$ L is a **ceiling, not a recoverable target**: the additive envelope S_{opt} is unrealisable (passes do not commute).

Publications

F. Bruzzone, W. Cazzola, M. Brancaleoni, and D. Pellegrino, *Sink or SWIM: Tackling Real-Time ASR at Scale*, IEEE Transactions on Audio, Speech and Language Processing (TASLPRO), 2026. DOI: 10.1109/TASLPRO.2026.3675780. (Q1 Journal)

F. Bruzzone, W. Cazzola, and L. Favalli, *Code Less to Code More: Streamlining Language Server Protocol and Type System Development for Language Families*, Journal of Systems and Software (JSS), vol. XXX, article 112554, 2025. DOI: 10.1016/j.jss.2025.112554. (Q1 Journal)

Submitted to A* Conferences:

[International Conference on Software Engineering (ICSE)] F. Bruzzone and W. Cazzola, *A Multi-Dimensional, Per-Pass Empirical Study of the LLVM Optimization Pipeline.*

Under review to Q1 Journals:

[Journal of Systems and Software (JSS)] — F. Bruzzone, W. Cazzola, and L. Favalli, *Generalized Software Product Line Extraction*, arxiv.org/abs/2605.28989, 2026.

[Transaction on Programming Languages and Systems (TOPLAS)] — F. Bruzzone, W. Cazzola, and L. Favalli, *Meta-Monomorphizing Specializations*, arxiv.org/abs/2602.12973, 2026.

[Transaction on Programming Languages and Systems (TOPLAS)] — F. Bruzzone, W. Cazzola, and L. Favalli, *From Separate Compilation to Sound Language Composition*, arxiv.org/abs/2602.03777, 2026.

[Journal of Systems and Software (JSS)] — F. Bruzzone, W. Cazzola, and L. Favini, *Prioritizing Configuration Relevance via Compiler-Based Refined Feature Ranking*, arxiv.org/abs/2601.16008, 2026.

PhD Courses and International Collaboration

I'm collaborating with MLIR and LLVM community.

I'm planning collaborate with:

- RWTH Aachen University
- ARM
- Roofline.ai

Courses taken (CFU 13/12):

- RESOURCES ALLOCATION IN MOBILE EDGE COMPUTING, INF/01, AP, 31-10-2024, CFU 2
- CONSTRUCTING AND MINING BIOMEDICAL KNOWLEDGE GRAPHS, INF/01, AP, 27-02-2025, CFU 4
- MATEURISTICHE PER PROBLEMI DI OTTIMIZZAZIONE COMBINATORIA (MODULE 1), INF/01, AP, 31-10-2025, CFU 2
- MATEURISTICHE PER PROBLEMI DI OTTIMIZZAZIONE COMBINATORIA (MODULE 2), INF/01, AP, 27-11-2025, CFU 2
- SHAPE YOUR OWN PROGRAMMING LANGUAGE, INF/01, AP, 19-03-2026, CFU 3